1. Analyze the two GenAI programs. The analysis must include:
   a. Name of two GenAI programs
   Claude
   ChatGPT

b. How you accessed them (e.g., Google query, CoPilot on my computer)
Both through google so the website for both of the AI's

c. Prompt used. (You must use the same prompt for both GenAIs.)
I used 2 - 3 images of the assignment requirements on the second page and then the test file format and the sample text.

d. Generated code from both GenAIs.
I cant paste both here because they are both about 500 lines of code and comments, but they are in the repo and easy to access if you have this pdf

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.
Claude

1. Correctness - It reproduces the sample output exactly. It also handles the edge cases that weren't in the last assignment. Two NEXTs in a row print identical output, because peekMax() never modifies the heap. Two READs in a row each silently remove the current top email. NEXT and READ on an empty inbox don't crash or error. Matching categories go to the newer date. Dates are converted to YYYYMMDD integers, so year and month compare correctly.
2. Execution Time - The priority rank and date key are computed once, so comparisons are integer comparisons rather than repeated string parsing. Which was the big efficiency decision. Inserting a new email with EMAIL costs O(log n). This is because the heap is a complete binary tree stored in an array, so its height is proportional to log n. The NEXT command costs O(1), constant time, because the highest-priority email is always sitting at index 0 of the underlying array. The READ command costs O(log n) for the same reason EMAIL does. The COUNT command costs O(1), since the heap already tracks its size as the length of the underlying vector, so no counting or traversal is needed.
3. Space Complexity - O(n) overall. Each email stores three strings plus two integers. priorityRank and dateKey are derived from the strings, so they're technically redundant. std::vector capacity can reach about 2× the element count after growth, and it never shrinks after READs. peekMax() returns a copy. Returning a const Email& would avoid that.
4. Maintainability - The heap logic (MaxHeap) is separate from the email data (Email). All priority logic lives in one method, hasHigherPriorityThan. Changing the rules, such as

adding a sender category, touches one place. Names are descriptive, the heap index math is isolated in helper methods, and every line is commented per the rubric.

ChatGPT

1. Correctness - The program correctly implements the required email priority system using a list-based MaxHeap. Each email is stored as an Email object containing the sender category, subject, and date. They are then assigned priorities in the order required in the instructions document. If the priority is the same then it converts to date and the newer email gets higher priority. NEXT displays the highest-priority email without removing it, so two consecutive NEXT commands produce the same email. READ removes the highest-priority email without requiring a previous NEXT, and COUNT reports the number of unread emails.
2. Execution Time - It has a efficient time because the heap doesn't need to work through the whole inbox when emails are added or removed. For EMAIL, the new email is initially added to the end of the vector and heapifyUp() moves it toward the root. A binary heap has a height of approximately log n, so insertion has a worst-case execution time of O(log n). For READ, the root is replaced with the final element and heapifyDown() restores the heap. The email can move through at most the height of the heap, giving READ a worst-case execution time of O(log n). For NEXT, the program accesses heap index 0, which is already the highest-priority email. This takes O(1) time. COUNT uses heap.size(), so it also takes O(1) time.
3. Space Complexity - The program has an overall space complexity of O(n), where n is the number of unread emails. Each Email stores three strings, the sender category, subject, and date. As additional emails are added, the amount of memory required grows based on how much text is in the email. The main data structure requires O(n) space and the heap maintenance algorithms require O(1) additional auxiliary space.
4. Maintainability - The program has good maintainability because it separates responsibilities into three classes. This separation makes changes easier because one part of the program can usually be modified without rewriting unrelated sections. For example, the sender priority rules are primarily contained in getSenderPriority(), while heap behavior is contained in MaxHeap. It also has a good amount of comments to help editors understands individual lines and functions.

f. Select one to use as the basis for this assignment and justify why you picked it.
Claude because it caches priorityRank and dateKey on each Email object at construction, so heap comparisons are integer comparisons rather than re-parsing the date string every time.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

1. Correctness - main now checks argc: if a filename is given it reads that file, otherwise it reads stdin. Strips \r from every line. A stripCarriageReturn helper removes a trailing carriage return before parsing. Added an arrivalSequence field, incremented on every EMAIL insert, used as a final tie-breaker when two emails share both category and date.splitCommaFields now returns false instead of producing garbage substrings if a line doesn't have two commas, and dateToKey throwing invalid_argument is caught.
2. Execution Time - peekMax() returns const Email& instead of a copy. The original copied three strings on every NEXT call. This is now a direct reference, so NEXT does zero string copying. extractMax() uses std::move instead of a plain assignment when moving the last element into the root position. The original copied three strings on every READ; this version transfers ownership instead.
3. Space Complexity - Still O(n) overall no change in what's stored per email. I kept the cached priorityRank/dateKey/arrivalSequence trade-off since that was already a reasonable design choice, not a flaw. Made Email's fields private with const string&-returning getters, replacing the direct public-field access in the original.
4. Maintainability - Split main()'s logic into a CommandProcessor class that owns the heap, parses lines, and dispatches to per-command handler methods. The original did all of this inline in main, mixing I/O, parsing, and dispatch in one function. Replaced the sender-category if/else chain with a lookup table so the priority order is defined in one readable place instead of scattered magic numbers

Command lines to run programs if needed

ChatGPT:
g++ -std=c++11 gptassignment3.cpp -o gptassignment3
./gptassignment3 input.txt

Claude:
g++ -std=c++11 claude_email_priority_queue.cpp -o claude_email_priority_queue
./claude_email_priority_queue < input.txt

Improved Claude:
g++ -std=c++11 claudeimproved_assignment3.cpp -o claudeimproved_assignment3
./claudeimproved_assignment3 input.txt